768 - Introducción a los Algoritmos y Flujo de Datos
Lectura - Vigente para el Segundo Semestre 2026

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas
Docentes: Edgar Francisco Rodas Robledo, Herman Igor Véliz Linares
Tutores Académicos: Emiliano José Alexander Velásquez Najera, Josue Daniel Chavez Portillo, Jorge Andrés Mejía Súchite

Cómo piensa una computadora: números y caracteres

Tiempo estimado: 4 min

Fecha programada: Semana 4 – 08/08/2026

Título del Documento / Capítulo	Cómo piensa una computadora: sistemas de numeración y codificación de caracteres
Autor(es)	Tutores Académicos – Laboratorio 768 Algoritmos (con base en Tanenbaum y Austin, 2013)
Tipo de Recurso	Documento Técnico

Índice

1. Marco Formativo.....2

1.1 Valores.....2

1.2 Competencia(s).....2

2. Palabras Clave.....3

3. Introducción.....3

4. Desarrollo del Tema.....3

4.1. Sistemas de numeración.....3

4.2. Métodos de conversión.....3

4.3. Codificación.....3

5. Orientación a Casos de Estudio.....3

5.1. Problema Real.....3

5.2. Solución Aplicada.....4

Figura 1 - Conversión por divisiones sucesivas y notación posicional.....4

5.3. Lecciones Aprendidas.....4

Tabla 1 - Sistemas de numeración.....5

6. Conclusiones.....5

7. Preguntas de Autoevaluación.....5

8. Referencias.....5

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Rigor y verificación	Verificar cada conversión numérica y cada codificación de caracteres paso a paso: un solo bit incorrecto cambia por completo el dato representado.

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Analiza problemas computacionales y formula soluciones algorítmicas utilizando pensamiento computacional, pseudocódigo y diagramas de flujo en las tecnologías establecidas para representar el procesamiento y flujo de datos bajo criterios de corrección lógica, claridad y orden
Competencia Específica	Convierte valores entre los sistemas decimal, binario, octal y

	hexadecimal aplicando métodos de conversión posicional y estándares de codificación de información para representar datos digitales con precisión y verificar los resultados obtenidos

2. Palabras Clave

Base: Cantidad de símbolos distintos de un sistema de numeración.

Bit / byte: Dígito binario / agrupación de 8 bits.

Notación posicional: El valor de un dígito depende de su posición.

ASCII: Código de 7 bits que asigna un número a cada carácter básico.

Unicode: Estándar universal que codifica los caracteres de casi todos los idiomas.

3. Introducción

Los circuitos electrónicos distinguen solo dos estados, por lo que toda la información (números, textos, imágenes) se representa con ceros y unos. Entender cómo se convierte entre bases y cómo se codifican los caracteres evita errores al trabajar con datos digitales.

Esta lectura resume los métodos de conversión y los estándares de codificación más usados.

4. Desarrollo del Tema

4.1. Sistemas de numeración

En un sistema posicional con base b, el número $d_2 d_1 d_0$ vale $d_2 \cdot b^2 + d_1 \cdot b + d_0$. Los sistemas más usados en computación son binario (2), octal (8), decimal (10) y hexadecimal (16).

4.2. Métodos de conversión

Desde decimal se aplican divisiones sucesivas entre la base; hacia decimal se usa la notación posicional. Entre binario, octal y hexadecimal basta agrupar los bits de 3 en 3 o de 4 en 4.

4.3. Codificación

ASCII asigna un código a cada carácter; ISO-8859 amplía el conjunto con letras acentuadas y Unicode unifica todos los idiomas (UTF-8). Las imágenes se representan como matrices de píxeles y el audio como muestras de la onda sonora.

5. Orientación a Casos de Estudio

5.1. Problema Real

Descripción del Contexto: Un sitio web de una universidad muestra «InscripciÃ³n» en lugar de «Inscripción» en el formulario de registro.

Impacto: Los estudiantes desconfían del sitio y los datos con tildes se almacenan corruptos en la base de datos.

5.2. Solución Aplicada

Marco Teórico / Metodología: El problema surge de interpretar los bytes de un archivo UTF-8 como ISO-8859-1. La solución es declarar y usar la misma codificación en todo el flujo.

Pasos de Implementación:

- Identificar la codificación real del archivo.
- Declarar la codificación (por ejemplo, UTF-8) en el servidor y en el HTML.
- Verificar con caracteres especiales (á, é, ñ).

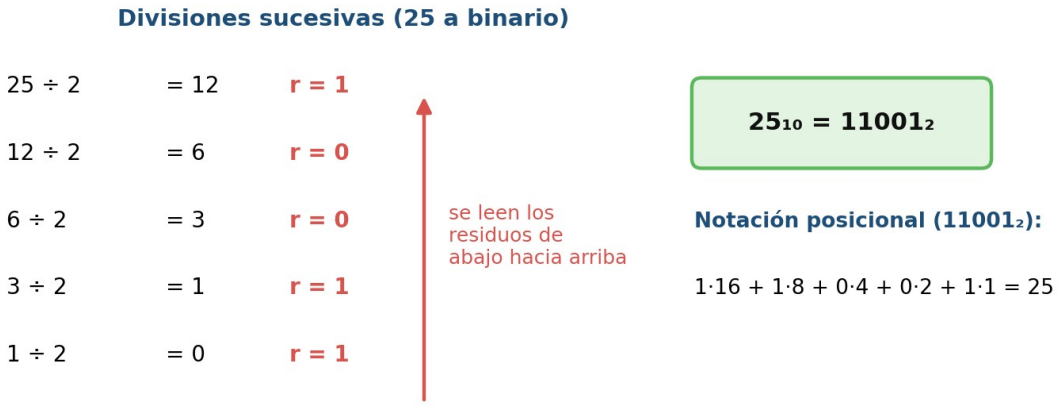


Figura 1 - Conversión por divisiones sucesivas y notación posicional

5.3. Lecciones Aprendidas

Resultados Clave: Con una codificación única, los textos se visualizan correctamente.

Consideraciones / Limitaciones: Los archivos antiguos pueden estar en ISO-8859; conviene convertirlos a UTF-8.

Sistema	Base	Símbolos	Uso
Binario	2	0, 1	Circuitos digitales
Octal	8	0–7	Permisos en Unix
Decimal	10	0–9	Uso cotidiano

Sistema	Base	Símbolos	Uso
Hexadecimal	16	0–9, A–F	Direcciones, colores

Tabla 1 - Sistemas de numeración

6. Conclusiones

- Los métodos de conversión permiten pasar entre cualquier base.
- ASCII y Unicode asocian números con caracteres.
- Verificar cada conversión evita errores de representación.

7. Preguntas de Autoevaluación

1. ¿Cuál es el factor principal que produce los caracteres extraños del caso?
2. Explique cómo se convierte un número decimal a binario.
3. ¿Por qué el hexadecimal es útil para representar binarios?
4. ¿Qué diferencia hay entre ASCII y Unicode?
5. ¿Cómo verificaría el resultado de una conversión?

8. Referencias

Tipo de Recurso	Descripción del Contenido	Enlace / Ubicación
Documentación Oficial (Si aplica)	Unicode Standard	https://www.unicode.org/standard/standard.html
Libro de Texto (Si aplica)	Tanenbaum y Austin (2013)	Libro de texto
Esquemas / Diagramas (Si aplica)	Figura 1 (elaboración propia)	Lectura S4
Repositorio de Código (Si aplica)	Tabla ASCII	https://www.ascii-code.com/